Abgabe 1: Einführung, GPIOs, LCD

Abzugeben sind bis Sonntag nach der mündlichen Abnahme, um 23:59 Uhr:

- eine in Ihrem Git Repository hinterlegte PDF-Datei mit dem Namen `abgabe1.pdf`, in der die unten *kursiv* angegebenen Fragen beantwortet werden. Sie haben hier die Freiheit, ein beliebiges Tool zur Erstellung der PDF zu benutzen, wobei wir Ihnen natürlich LaTeX empfehlen würden.
- der in Ihrem Git Repository mit dem Tag `abgabe_1` eingecheckte Code in den CubeIDE-Projekten **00_Introduction**, **01_ESD**, **02_Joystick** und **03_LCD** zu den praktischen Aufgaben (nicht kursiv).

Entwicklungssystem (CMSIS, Cube)



Sie Finden Hinweise zu diesen Aufgaben in den Quellen [1], [2], [3]

1. *Welche Grundarchitektur besitzt der Cortex-M4?*
2. *Wozu dient der DMA-Controller?*
3. *Welche Vorteile bietet der so genannte "Thumb2" Befehlssatz?*
4. *Besitzt unser Controller eine FPU? Beschreiben Sie auch, wozu diese dient.*
5. *Welche Vorteile bietet CMSIS für den Entwickler?*
6. *Warum vereinfacht CMSIS die Portierung von Software auf unterschiedliche Mikrocontroller?*
7. *Beschreiben Sie das Cube Ökosystem, von dem die CubeHAL auch nur ein Bestandteil ist. Recherchieren Sie hierzu auf der Website von STMicroelectronics.*

Arbeitsweise mit der CubeIDE

Finden Sie heraus, wie Sie mit dem Debugger die folgenden Jobs erledigen können und dokumentieren Sie dies kurz.

1. *Setzen von Breakpoints (Haltepunkte) und Watchpoints (Beobachtungspunkten). Welche Funktion erfüllen Sie?*
2. *Schrittweise Ausführung von Programmcode mit und ohne Überspringen von Funktionsaufrufen*
3. *Arbeiten mit Variablen und Ausgabe ihres Inhalts.*
4. *Arbeiten mit Registern des Prozessors und der Peripherien, sowie Ausgabe ihres Inhalts.*

LED_on

1. *Wie oben erläutert, ist die grüne LED an PG13 angeschlossen. An welchen Pin ist die rote LED des Discovery Boards angeschlossen?*
2. Konfigurieren Sie die GPIO-Ports für den Betrieb beider LEDs.
3. Aktivieren Sie nun die grüne LED.

4. Aktivieren Sie mithilfe eines einzigen Funktionsaufrufs beide LEDs.
5. Aktivieren Sie beide LEDs und deaktivieren Sie anschließend die rote LED wieder.
6. Deaktivieren Sie mit nur einem Funktionsaufruf beide LEDs.
7. Aktivieren Sie die grüne LED und togglen Sie daraufhin beide LEDs mit der Toggle Funktion. Rufen Sie diesen Befehl mehrmals auf, um den Effekt zu sehen.



Nutzen Sie bei diesen Aufgaben den Debugger, um im Einzelschritt-Modus die Auswirkung der einzelnen Schreibbefehle zu beobachten. Setzen Sie zunächst einen Break-Point, indem Sie den ersten interessanten Befehl markieren und im Kontextmenü (rechte Maustaste) Insert Break-Point wählen. Das Programm läuft nach dem Start nun zunächst bis zu diesem Break-Point. Ab hier können Sie im Einzelschritt-Modus (Step over) fortfahren. Mittels Step over können Sie verhindern, dass der Debugger in eine Unterroutine hinein springt.

8. *Finden Sie heraus, welche GPIO_Modes es neben dem Mode* `GPIO_MODE_OUTPUT_PP` *noch gibt.*



Klicken Sie im Quellcodeeditor mit der rechten Maustaste auf `GPIO_MODE_OUTPUT_PP` und wählen Sie im Kontextmenü **Open Declaration** (oder klicken Sie mit der linken Maustaste drauf, während Sie **Strg** gedrückt halten).

9. *Lesen Sie sich das Kapitel 8 aus dem Reference Manual des STM32F429 Prozessors durch* [4]*, um einen Überblick über die Funktionalität der GPIOs zu erhalten. Vergleichen Sie die in der HAL angebotenen Ausgabe-Modi mit den im Reference Manual dokumentierten Modi.*

GPIO: Digitale Ausgabe

1. *Dokumentieren Sie in einer Tabelle die feste Zuweisung von GPIO-Pins zu 8-Segment-Leitungen, um im Folgenden nicht mehr die verschiedenen Schaltpläne abgleichen zu müssen.*
2. Um die Übersichtlichkeit im Sinne einer sauberen Programmstruktur zu wahren, sollen Sie angefangen bei der Ansteuerung des 8-Segment-Displays Ihre Software mithilfe von Modulen strukturieren. Erstellen Sie dafür zunächst einen Ordner "esd" (eight segment display) in dem schon vorhandenen Ordner "modules". Innerhalb dieses Ordners erstellen Sie zwei Dateien, "esd.h" und "esd.c". Dazu klicken Sie mit der rechten Maustaste auf den Ordner "esd" und wählen dann *New → Source File* und *New → Header File*.



Vergessen Sie nicht in ihrem Modul mittels `#include "stm32f4xx.h"` die Header-Datei der STM32Cube HAL zu inkludieren, da Sie später Funktionen aus dieser verwenden möchten.

Legen Sie anschließend in der Datei "esd.h" Deklarationen für die folgenden Typdefinitionen und Funktionen an:

```
typedef enum {
    ESD_DIGIT_0,
    ESD_DIGIT_1,
    ESD_DIGIT_2,
```

```
    ESD_DIGIT_3,
    ESD_DIGIT_4,
    ESD_DIGIT_5,
    ESD_DIGIT_6,
    ESD_DIGIT_7,
    ESD_DIGIT_8,
    ESD_DIGIT_9
} esd_digit_t;

typedef enum {
    ESD_POSITION_1,
    ESD_POSITION_2,
    ESD_POSITION_3,
    ESD_POSITION_4,
    ESD_POSITION_ALL
} esd_position_t;

void esd_init(void);
void esd_show_digit(esd_digit_t digit, esd_position_t pos);
```

Als nächstes inkludieren Sie in "esd.c" die Header-Datei "esd.h" und erstellen die zugehörigen Funktionsrümpfe.



Includieren sie **keine** *.c Dateien innerhalb anderer Header- oder Sourcecode-Dateien. Dies wäre ein extrem schlechter Programmierstil und widerspricht der Philosophie der Programmiersprache C! Eigenständige Programmmodule werden in C immer erst in Form ihrer kompilierten Objektcode-Dateien vom sog. Linker (dt. Binder) zum fertigen Programm zusammengefügt. Dies beschleunigt erheblich den Kompilationsprozess des gesamten Projekts, da nicht jedes Modul bei einer Änderung eines anderen Moduls neu gebaut werden muss.

3. Implementieren Sie die vorgegebene Funktion `esd_init`, indem Sie Ihr Wissen über GPIOs aus Kapitel **LED_on** der Praktikumsanleitung anwenden, um die benötigten 13 GPIO-Pins für die Ansteuerung des 8-Segment-Boards zu initialisieren.
4. Implementieren Sie die vorgegebene Funktion `esd_show_digit`, welche sowohl die Ziffer, als auch die Position, an der sie angezeigt werden soll, entgegennimmt.
5. Nun geht es um die Benutzung Ihres neuen Moduls für das Erstellen eines Programms. Inkludieren Sie dafür `esd.h` in der `main.c`, um nun mithilfe der in den obigen Aufgaben erstellten Funktionen einen Countdown zu implementieren. Dieser soll im 1-Sekunden-Takt von 9 bis 0 runterzählen und diesen Counter-Wert auf der ersten Position des 8-Segment-Boards anzeigen. Ist er bei 0 angekommen, soll der Countdown auf der nächsten Position automatisch wieder bei 9 beginnen. Dies wiederholt sich bis der Countdown bei der letzten Position durchgelaufen ist. Dann soll er nochmal auf allen Ziffern gleichzeitig herunter zählen, woraufhin er wieder bei der ersten Position beginnt.

   Um das Programm nach dem Anzeigen einer Ziffer für eine Sekunde ausbremsen zu können, erstellen Sie analog zu dem Modul *esd* ein neues Modul namens *utils*, welches eine Funktion

`utils_delay_ms(uint32_t t)` enthalten soll. Diese soll eine zeitliche Verzögerung für die übergebene Anzahl von `t` Millisekunden umsetzen.

6. *Wie können Sie dies erreichen? Dokumentieren Sie die Möglichkeiten, die Ihnen einfallen, um nur mithilfe von Software eine Sekunde zu warten.*



Bietet die STM32Cube HAL eine Funktion für solche Zwecke? Schauen Sie im CubeIDE-Projekt unter **HAL_Driver/Inc/stm32f4xx_hal.h** nach. Sollten Sie auf diese Funktion zurückgreifen, müssen Sie erklären können, wie diese funktioniert und wovon ihre Funktionsweise abhängt!



Nutzen Sie das neue Modul *utils* auch in Zukunft für allerlei Funktionen, die sinngemäß keinem einzelnen Modul zugewiesen werden oder aufgabenübergreifend genutzt werden können.

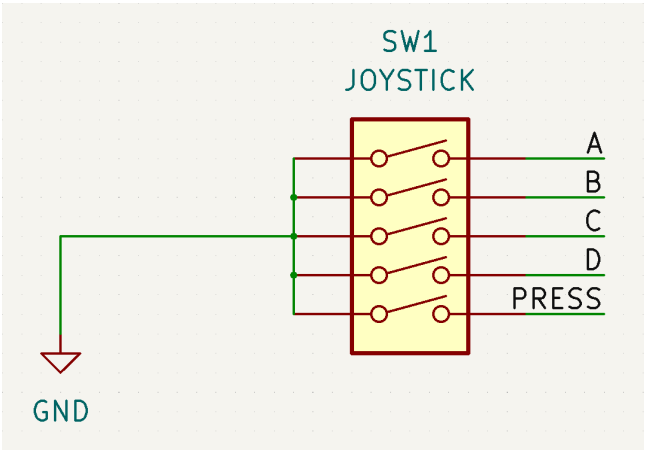
7. Gibt es eine Möglichkeit, mit einem Befehl mehrere Portpins zu setzen **und gleichzeitig** zu löschen und somit die Binärdarstellung einer Zahl auf dem GPIO Port zu übernehmen? Wenn ja, schreiben Sie dafür eine Funktion `utils_gpio_port_write`, welche den gewünschten Port sowie das zu schreibende Bitmuster als Parameter erwartet.



Die Port-Angabe in der HAL (z.B. der erste Parameter der schon kennengelernten Funktion `HAL_GPIO_WritePin`) geschieht mithilfe der Strukturdefinition `GPIO_TypeDef`. Schauen Sie sich diese Strukturdefinition in dem Projekt unter **CMSIS/device/stm32f429xx.h** an und lesen Sie in Kapitel 8.4 des Reference-Manuals [4] nach, welche Bedeutung die darin enthaltenen Register haben.

GPIO: Digitale Eingabe

1. *An welche GPIO-Pins ist der Joystick angeschlossen? Betrachten Sie dafür die Beschriftung des Joysticks und der daneben liegenden Jumper!*
2. *Welcher GPIO-Mode muss für die Pins eingestellt werden, damit digitale Signale eingelesen werden können?*
3. *Betrachten Sie den folgenden Ersatzschaltplan des Joysticks. Muss für die Pins ein Pull-Up- oder ein Pull-Down-Widerstand aktiviert werden?*



4. *Müssen Sie die Speed-Komponente der Initialisierungsstruktur füllen? Betrachten Sie hierzu im Reference Manual [4] Tabelle 36.*
5. *Welche Funktion bietet die HAL, um Daten von einem GPIO-Port einzulesen?*
6. Orientieren sich an dieser Lese-Funktion, um entsprechend der oben von Ihnen erstellten Funktion `utils_gpio_port_write` eine weitere namens `utils_gpio_port_read` zu schreiben, welche den zu lesenden Port als Parameter erwartet und dessen aktuellen Inhalt als Bitmuster zurückgibt.
7. Schreiben Sie analog zu dem Modul *esd* ein neues Modul *joystick*, welches die Initialisierung des Joysticks und das Auslesen der aktuellen Joystick-Richtung implementiert. Nutzen Sie geeignete Definitionen und Makros für die verwendeten Pins.



Aktivieren Sie nur die GPIO-Pins, die Sie für die Verwendung des Joysticks benötigen.

8. Benutzen Sie die Module *esd* und *joystick*, um ein Programm mit folgenden Anforderungen umzusetzen:
   - Initial wird die Ziffer 0 auf der ersten Position des 8-Segment-Displays angezeigt.
   - Drückt man den Joystick hoch oder runter, so wird die Ziffer inkrementiert bzw. dekrementiert. Drückt man bei einer 9 ein weiteres Mal hoch, so erscheint die 0. Drückt man bei einer 0 ein weiteres Mal runter, so erscheint die 9.
   - Drückt man den Joystick nach rechts oder links, so wird die Position der aktuellen Ziffer entsprechend nach rechts oder links verschoben. Wird die Ziffer auf der vierten Position angezeigt und man drückt ein weiteres Mal nach rechts, so springt sie wieder auf die erste Position. Entsprechend springt die Ziffer auch bei der ersten Position und einem weiteren Drücken nach links auf die vierte Position.
   - Drückt man den Joystick in die Mitte hinein, so startet auf der aktuellen Position ein Countdown beginnend bei der aktuellen Ziffer. Steht beispielsweise aktuell eine 6 auf der dritten Position, so läuft der Countdown im 1-Sekunden-Takt von 6 bis 0, woraufhin wieder die 6 angezeigt wird und auf neue Joystick-Eingaben gewartet wird.
9. *Fällt Ihnen bei dem Betätigen des Joysticks im Bezug auf Ihr Programmverhalten etwas ungewöhnliches auf? Hierbei handelt es sich um einen bekannten mechanischen Störeffekt von Tastern. Wie nennt sich dieser Störeffekt und welche Möglichkeiten existieren, um diesem entgegenzuwirken?*
10. (Zusatzaufgabe) *Zeichnen Sie zu der in Aufgabe 8 implementierten Anwendung ein Zustandsdiagramm (State-Chart), welches den Ablauf des Programms inklusive Ereignisse und Aktionen während den Zustandsübergängen darstellt.*

LCD

1. *Welche Funktionen gibt es, um Text auf dem Display auszugeben? Worin besteht der Unterschied? Erklären Sie die Parameter, die die Funktionen erwarten.*



Erstellen Sie für die folgenden praktischen Aufgaben ein neues Modul *my_lcd*, welches die bereitgestellte Bibliothek `lcd` inkludiert und vom Hauptprogramm in der `main.c` benutzt wird.

2. Um Zahlen auszugeben bedienen Sie sich der Funktion `sprintf()`. Binden Sie dafür die Header-Datei `<stdio.h>` ein. Legen Sie sich nun einen Puffer an und füllen Sie ihn mithilfe der `sprintf()` -Funktion. Geben Sie den Puffer mit einer der Textanzeige-Funktionen aus. Hier ein Beispiel:

```
#include <stdio.h>
...
char buf[64];
...
sprintf(buf, "Zahl = %d", 123);
lcd_draw_text_at_line(buf, 4, BLACK, 2, WHITE);
```



Auf Mikrocontrollern ist der verfügbare RAM-Speicher und die Stackgröße relativ beschränkt. Daher sollte der Puffer nur so groß wie tatsächlich nötig gewählt werden. Wenn Sie bezüglich der zu schreibenden Bytes in den Puffer auf Nummer sicher gehen wollen, können Sie auch die Funktion snprintf verwenden.

3. Programmieren Sie einen Countdown, der im Takt von ca. einer Sekunde von 10 bis 1 zählt.
4. Benutzen Sie die `ILI9341_Draw_Filled_Rectangle_Coord`-Funktion, um die Funktion `my_lcd_draw_baargraph` zum Zeichnen eines horizontalen Bargraphs zu schreiben. Diese nimmt die Parameter `x`, `y`, `width`, `height`, `value`, `color` und `bgcolor` entgegen. `x` und `y` geben die Startkoordinaten in der unteren, linken Ecke des Bargraphs an. `width` und `height` die Länge und Breite des Bargraphs. `value` soll ein Wert in Promille (zwischen 1 und 1000) sein und gibt an, wie viel Anteil des Bargraphs proportional zum Wert `width` mit der Farbe `color` gefüllt werden soll. Der **Rest** wird mit der Farbe `bgcolor` gefüllt. Denken Sie daran, die Namen der Parameter dieser Funktion mit dem entsprechenden Datentyp, den Sie gewählt haben, zu prefixen.
5. Zeigen Sie nun zusätzlich den Wert ihres Countdowns mit Hilfe der Bargraph-Anzeige an.
6. Benutzen Sie die `lcd_draw_pixel`-Funktion und Zeichnen Sie damit ein Kreuz mit den Maßen 100x100 Pixeln.

[1] Diese finden Sie im selben Repository, wo Sie auch die Praktikumsanleitung und diese Aufgaben her haben unter **Begleitende_Literatur/Kendl_CMSIS.pdf**

[2] Im selben Repository unter **Begleitende_Literatur/Pfaffelmoser_Diplomarbeit.pdf**

[3] https://en.wikipedia.org/wiki/ARM_Cortex-M

[4] Im Repository unter **Resourcen/STM32F429-Reference-Manual**